

```

}

public Date getSince() {
    return since;
}
}
}

Following is the simple InvalidUserException class that
merely extends the system defined Exception class:

```

```

package com.glarimy.ums.domain;

@SuppressWarnings("serial")
public class InvalidUserException extends Exception{

    public InvalidUserException() {
        super();
        // TODO Auto-generated constructor stub
    }

    public InvalidUserException(String message, Throwable cause,
        boolean enableSuppression,
        boolean writableStackTrace) {
        super(message, cause, enableSuppression,
            writableStackTrace);
        // TODO Auto-generated constructor stub
    }

    public InvalidUserException(String message, Throwable cause)
    {
        super(message, cause);
        // TODO Auto-generated constructor stub
    }

    public InvalidUserException(String message) {
        super(message);
        // TODO Auto-generated constructor stub
    }

    public InvalidUserException(Throwable cause) {
        super(cause);
        // TODO Auto-generated constructor stub
    }
}

```

The *UserRepository* interface just offers the contract for the repository. Since the *AddService* only saves the user data, the repository is sufficient to offer only that one operation.

```

package com.glarimy.ums.domain;

public interface UserRepository {

```

```

    public User save(User user);
}

```

The *UserRepositoryImpl* somehow saves the user data using technology-specific implementation. For example, it may use JDBC to save data in an RDBMS. We will discuss the data layer at length in future.

```

package com.glarimy.ums.data;

import com.glarimy.ums.domain.User;
import com.glarimy.ums.domain.UserRepository;

public class UserRepositoryImpl implements UserRepository {

    @Override
    public User save(User user) {
        // TODO
        return user;
    }
}

```

The application layer

This layer consists of DTOs, data mappers, and controllers. The application layer acts as the entry point to the domain service. In other words, it accepts the user requests and passes them to the domain layer.

In the case of *AddService*, the only kind of request that is accepted is to add a new user. The *UserController* offers the *add()* method for this purpose. It accepts a *NewUser* as the parameter and returns *UserRecord*. Both the *NewUser* and *UserRecord* are DTOs. Refer to Figure 4.

A DTO is just a data structure useful for carrying the data between the domain layer and the client. It does not carry any domain logic.

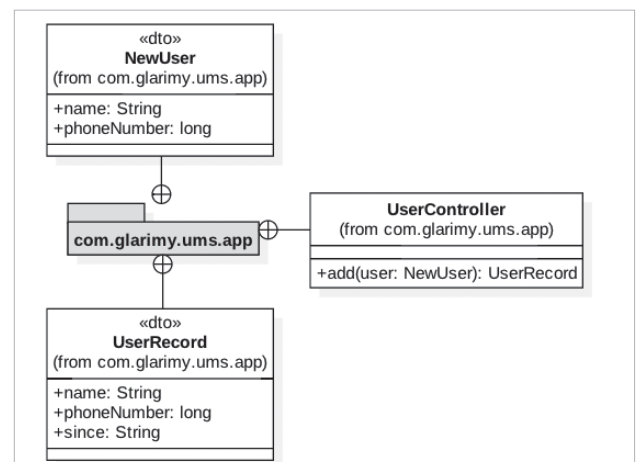


Figure 4: The application layer of *AddService*